Float — Product Requirements Document

Personal cash-flow planning · Nov 3, 2026 · @Tats

1. Product definition

Float is a mobile-first personal cash-flow forecasting PWA: it turns balances, expected income and upcoming commitments into a day-by-day projection, and from that a Safe-to-Spend Number users can inspect line by line.

Vision

People with irregular income always know what they can safely spend today, and why.

One-line pitch: Your bank balance tells you what you have. Float tells you what you can spend.

Portfolio blurb: Float is a cash-flow planning PWA for people whose income doesn't arrive on a fixed payday. It separates total, reserved and available money, projects balances 90 days ahead from recurring and one-off income and expenses, and calculates a transparent Safe-to-Spend Number. A what-if simulator tests decisions without touching real data, and AI explains the forecast in plain language while a pure, fully tested TypeScript engine does every calculation.

Problem statement

Budgeting apps answer "where did my money go?"; people with irregular income need to know how much they can spend today without running out before the next payment.

A freelancer with ₦500,000 in the bank has rent of ₦200,000 due in two weeks, electricity and subscriptions coming, ₦30,000 owed to a friend, and a client payment with no firm date. Their spendable amount may be closer to ₦150,000, or less if the client is late. Most people work this out in their heads or a notes app and get it wrong: overspending and missing rent, or underspending from anxiety.

Budgeting apps track categories and the past; bank apps show one balance; spreadsheets can forecast but break under recurring rules, uncertain income and "what if" questions.

Product objectives

ID	Objective	Measured by
OBJ-1	Reach a first trustworthy Safe-to-Spend Number quickly	< 15 min from sign-up

ID	Objective	Measured by
OBJ-2	Make every number inspectable	100% of headline figures open a breakdown
OBJ-3	Keep the forecast honest about uncertainty	Uncertain income never counted in Safe-to-Spend
OBJ-4	Let users test decisions without risk	Scenarios never modify real data unless applied
OBJ-5	Make the daily check a 10-second phone habit	Home screen interactive in < 2 s on a mid-range phone

Value proposition and differentiation

- **For freelancers and commission earners:** a spendable number that accounts for rent, bills and late clients.
- **For anyone saving for something:** reserved money stops looking spendable.
- **For people with dollar income:** naira bills and USD income in one view.

Float	Typical alternatives
Forward-looking timeline and 90-day projection as the main screen	Backward-looking spending charts
Total, reserved and available shown everywhere	One balance
Income certainty (confirmed, estimated, uncertain) changes the forecast	All expected income treated as certain
What-if scenarios layered over real data	Manual spreadsheet copies
Inspectable Safe-to-Spend with stated assumptions	Opaque "you can spend X" figures

Design principles: show the reasoning; pessimistic by default; simulations never touch real data; money is exact; guidance, not advice.

Target audience and personas

Freelancers, remote workers, self-employed and commission-based workers, people with irregular salaries or several income sources.

Persona	Situation	Needs	Pain points today
Tolu, freelance designer	3–4 clients paying on different dates; yearly rent	Know what's safe when a client is late	Spends rent money, then scrambles
Kelechi, remote developer	Paid monthly in USD; bills in naira	See both currencies; plan conversions	Converts too early or too late
Fatima, sales rep	Base salary plus variable commission	Count base as confirmed, commission as estimated	Plans around her best month
Musa, small business owner	Daily cash income; personal and business mixed	Separate accounts; reserve for suppliers	Can't tell business float from personal money
Ngozi, salaried with side income	Salary, side gigs, subscriptions, a loan	Avoid mid-month squeezes	Big bills cluster in one week

Success metrics

Metric	Target
Named engine scenarios passing (including the worked example)	100%
Engine property tests over thousands of random cases	0 failures
Time from sign-up to first Safe-to-Spend	< 15 min
Simulator recalculation while typing	< 50 ms
Testers with irregular income using it for 2 weeks	10
Testers who can explain how Safe-to-Spend is calculated	8 of 10

Assumptions and decisions

#	Assumption or decision	Reason
D1	Safe-to-Spend always uses Cautious mode (confirmed income only), whatever forecast mode the chart shows	A spendable number must not depend on money that may not come
D2	The headline is the total Safe-to-Spend; the per-day figure is secondary	Totals are easier to act on; per-day can mislead with lumpy spending
D3	Accounts are required (data syncs across devices); a guest demo profile needs no sign-up	Cross-device use and backups matter for financial data
D4	No live bank connections; manual balances and CSV import only	The brief says not to assume integrations
D5	Exchange rates are entered by the user (with date); no live feed in the MVP	Avoids a paid API; rates are always labelled with their source and date
D6	Forecast horizon is 90 days	Matches the brief's milestones; longer horizons are mostly noise for irregular income
D7	AI explains and helps build scenarios; the engine computes every number	Shared AI rule: AI never replaces the calculation engine
D8	Float gives no investment, tax or credit advice	Guidance, not advice

Challenged from the brief: "Variable expenses" are modelled as estimated recurring amounts rather than a separate type; "insights" are rule-based and computed, with AI only rephrasing them.

2. Functional requirements

Requirements use the ID pattern FLT-<module>-<number> with a MoSCoW priority and testable acceptance criteria.

Core concepts

Term	Meaning
Total balance	Sum of included account balances in the base currency.

Term	Meaning
Reserved	Money set aside for named commitments; still in the account, not spendable.
Available	Total minus reserved.
Projected balance P(d)	Available on a future date after expected income and scheduled expenses up to that date.
Safe-to-Spend	The most you can spend today without projected available money falling below your buffer within 90 days, counting confirmed income only.
Account	Bank, savings, cash, foreign-currency or other, with its own currency.
Transaction	Something that happened: income, expense, transfer, adjustment.
Scheduled item	An expected income or expense with a date or recurrence rule.
Occurrence	One dated instance of a scheduled item, with its own status.
Certainty	Confirmed, estimated or uncertain (income only).
Commitment	A reservation toward an obligation, optionally linked to an expense.
Scenario	A list of hypothetical changes layered over real data.

User stories

ID	As a…	I want to…	So that…
US-01	User	add accounts with currency and balance	Float knows what I have
US-02	User	import transactions from a bank CSV	I don't type everything
US-03	User	add income with a schedule and certainty	the forecast reflects reliability

ID	As a…	I want to…	So that…
US-04	User	add recurring and one-off expenses	bills are never a surprise
US-05	User	reserve money for commitments	rent money stops looking spendable
US-06	User	see total, reserved and available together	I know my real position
US-07	User	scroll a timeline of money in and out	I see how my balance moves
US-08	User	see projected balances at 7, 14, 30, 60, 90 days	I can plan
US-09	User	be warned before I cross my buffer	I act early
US-10	User	see Safe-to-Spend and its breakdown	I trust it
US-11	User	try a purchase, delay or extra income safely	I decide without risk
US-12	User	mark items received, paid, skipped or late	the forecast stays accurate
US-13	User	correct a balance to match my bank	Float stays in sync

2.1 Accounts and transactions (ACC)

Stories: US-01, US-02, US-13.

ID	Requirement	Priority	Acceptance criteria
FLT-ACC-01	Create accounts: name, type, currency, opening balance	Must	Opening balance recorded as the first adjustment transaction
FLT-ACC-02	Transaction history per account (income, expense, transfer, adjustment)	Must	Account balance always equals the sum of its transactions
FLT-ACC-03	Balance correction records the difference as an adjustment	Must	Entering ₦412,300 when Float shows ₦415,000 creates a −₦2,700 adjustment

ID	Requirement	Priority	Acceptance criteria
FLT-ACC-04	Transfers between accounts, including cross-currency with the rate used	Must	Creates two linked transactions atomically; totals unchanged for same-currency transfers
FLT-ACC-05	CSV import with mapping, format detection, preview and duplicate detection	Must	Rows with the same date, amount and description as existing ones are flagged in preview
FLT-ACC-06	Exclude an account from forecasts	Must	Excluded balances leave totals and Safe-to-Spend
FLT-ACC-07	Saved import templates for common bank exports	Should	Re-importing the same bank's CSV pre-fills mapping

Rules: amounts are integer minor units. **Validation:** currency ISO 4217; name 1–60 characters. **Errors:** CSV unreadable; account currency changed after transactions exist (blocked). **Edge cases:** archiving an account with reservations (reservations must be moved first). **Dependencies:** FX rates (D5).

2.2 Income (INC)

Stories: US-03, US-12.

ID	Requirement	Priority	Acceptance criteria
FLT-INC-01	Income sources: name, amount, account, schedule, certainty	Must	Saved source appears on the timeline within the 90-day horizon
FLT-INC-02	Schedules: daily, weekly, biweekly, monthly (date or last working day), custom dates, one-time	Must	Each schedule type has recurrence tests including month ends
FLT-INC-03	Certainty per source and per occurrence	Must	Changing one occurrence to uncertain doesn't change the source
FLT-INC-04	Mark received (actual amount and date), late (new date) or missed	Must	Received creates a transaction; missed removes it from all forecasts
FLT-INC-05	Amount ranges forecast at the low end	Should	₦150k–₦250k counts as ₦150k in Expected mode

ID	Requirement	Priority	Acceptance criteria
FLT-INC-06	Overdue unmarked income auto-downgraded to uncertain	Must	One day after its date, the occurrence is excluded from Expected mode and flagged

Validation: amount > 0; end date after start. **Edge cases:** irregular income entered as custom dates; a one-time payment that arrives in two parts (record two receipts against one occurrence).

2.3 Expenses (EXP)

Stories: US-04, US-12.

ID	Requirement	Priority	Acceptance criteria
FLT-EXP-01	Expenses: one-time, recurring, subscription, debt repayment, bill, planned purchase; estimated amounts for variable bills	Must	Each kind can be created with amount, category, account, due date, recurrence, status, notes
FLT-EXP-02	Statuses: upcoming, paid, skipped, overdue	Must	Paid creates a transaction; overdue stays in the forecast on today's date
FLT-EXP-03	Pause a recurring expense or end it on a date	Must	Paused occurrences disappear from the forecast until resumed
FLT-EXP-04	Price change from a date (for example a subscription increase)	Should	Occurrences after the date use the new amount

Edge cases: expenses on the 31st fall on the last day of shorter months; a missed payment stays overdue until paid or skipped.

2.4 Commitments (COM)

Stories: US-05, US-06.

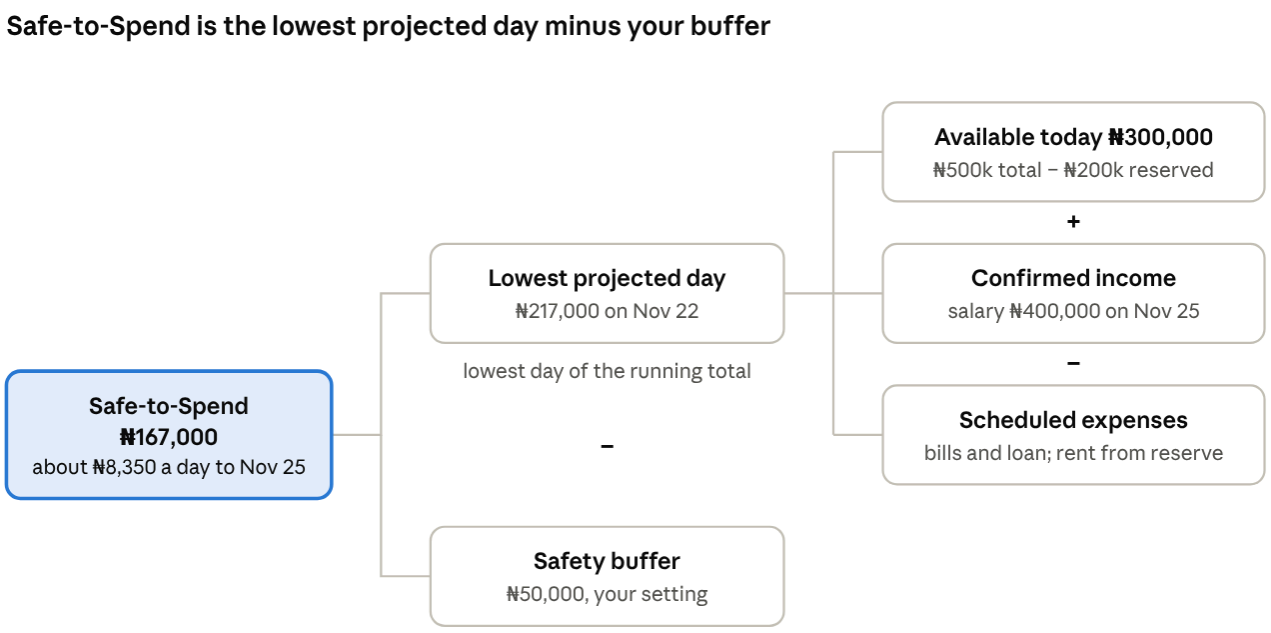
ID	Requirement	Priority	Acceptance criteria
FLT-COM-01	Reserve an amount for a named commitment from one or more accounts	Must	Available drops by the reserved amount immediately; total is unchanged

ID	Requirement	Priority	Acceptance criteria
FLT-COM-02	Link a commitment to an expense; paying it consumes the reservation	Must	Paying ₦200,000 rent from a ₦200,000 reservation leaves available unchanged
FLT-COM-03	Top up, reduce or release	Must	Each change is recorded with date
FLT-COM-04	Total, reserved and available on home, every account and the timeline	Must	All three visible without scrolling on a 375 px screen
FLT-COM-05	Goal-style reservations (reserve ₦20,000 every payday until ₦200,000)	Could	Phase 2

Edge cases: reserving more than an account holds (allowed, warned; account shows negative available); changing a future commitment applies from the chosen date only.

2.5 Timeline, forecasting and Safe-to-Spend (FCT)

Safe-to-Spend is the lowest point projected available money reaches in the next 90 days, minus the buffer, because money spent today lowers every future day by the same amount. Stories: US-07–US-10.



Safe-to-Spend calculation · worked example below

Read it right to left: the running total of available money, confirmed income and expenses sets the lowest day; the buffer comes off that.

How the forecast is built

1. **Expand** every income and expense rule into dated occurrences for 90 days in the user's timezone.
2. **Filter by mode:** Cautious (confirmed only), Expected (+ estimated), Optimistic (+ uncertain). Safe-to-Spend always uses Cautious (D1).
3. **Convert** to the base currency at the user's stored rate for that currency.
4. **Remove double counting:** expenses paid from reservations don't reduce available again; transfers between included accounts cancel out.
5. **Accumulate** a running projected balance per day from available today.

$$\text{Available}_0 = \text{Total} - \text{Reserved} \qquad P(d) = \text{Available}_0 + \sum_{t \le d} \text{Income}_t - \sum_{t \le d} \text{Expense}_t$$

$$\text{SafeToSpend} = \max\Big(0,\; \min_{0 \le d \le 90} P(d) - \text{Buffer}\Big)$$

Worked example (Nov 5, buffer ₦50,000)

Date	Item	Amount	Projected available
Nov 5	Total ₦500,000 minus rent reserved ₦200,000		₦300,000
Nov 7	Electricity	−₦20,000	₦280,000
Nov 10	Freelance payment (estimated, excluded in Cautious)	+₦150,000	₦280,000
Nov 15	Subscription	−₦8,000	₦272,000
Nov 19	Rent (paid from reservation)	−₦200,000	₦272,000
Nov 20	Internet	−₦25,000	₦247,000
Nov 22	Loan repayment	−₦30,000	₦217,000
Nov 25	Salary (confirmed)	+₦400,000	₦617,000

Lowest point ₦217,000 on Nov 22 → Safe-to-Spend **₦167,000**, about ₦8,350 a day for the 20 days to Nov 25.

ID	Requirement	Priority	Acceptance criteria
FLT-FCT-01	Chronological timeline from today: date, item, amount, running projected available	Must	Running balance on the last row equals P(90) from the engine
FLT-FCT-02	Uncertain income shown greyed and excluded from the running balance by default	Must	Toggling the mode updates the running balance
FLT-FCT-03	Days below buffer or below zero highlighted	Must	Highlight uses icon and label as well as colour
FLT-FCT-04	90-day chart of projected available with buffer line	Must	Readable at 375 px; tapping a point shows that day's items
FLT-FCT-05	Projected balances at 7, 14, 30, 60, 90 days plus lowest point and date	Must	Values equal P(d) from the engine
FLT-FCT-06	Buffer setting and alerts when the forecast crosses it	Must	Alert sent once per crossing date, by push or email
FLT-FCT-07	Safe-to-Spend on home: total, then per-day until next confirmed income	Must	Matches the worked example fixture exactly
FLT-FCT-08	Breakdown screen: starting available, included items, lowest day, buffer, excluded income with reasons, rates and assumptions	Must	Every figure on the breakdown sums to the headline
FLT-FCT-09	Labelled as an estimate based on the user's data, not financial advice	Must	Label visible on home and breakdown
FLT-FCT-10	Recalculate instantly on any change	Must	< 50 ms for 20 accounts and 200 rules

Rules: forecasts are computed, never stored as truth. **Edge cases:** no confirmed income in 90 days (per-day figure hidden; note shown); Safe-to-Spend floors at ₦0 with the shortfall named.

2.6 What-if simulator (SIM)

A scenario is a list of changes applied to a copy of real data at calculation time; the same engine runs on both. Stories: US-11.

ID	Requirement	Priority	Acceptance criteria
FLT-SIM-01	Change types: one-off expense, extra income (one-off or recurring), delay income, remove income, skip or remove expense, change recurring amount, change reservation, change buffer or mode	Must	Each type has an engine test comparing scenario vs real forecast
FLT-SIM-02	Comparison view: both lines on the chart, deltas for Safe-to-Spend, milestones and lowest point	Must	Deltas equal scenario minus real
FLT-SIM-03	Verdict sentence from rules: affordable, affordable but close to buffer (within 20%), not affordable with first problem date	Must	On the worked example, a ₦120,000 phone today returns "affordable" (lowest point ₦97,000); a ₦200,000 laptop today returns "not affordable", first below buffer on Nov 20
FLT-SIM-04	Discard, save, or apply to real data with a confirmation list	Must	Applying creates exactly the listed edits; nothing else changes
FLT-SIM-05	Saved scenarios re-run against current data; references to deleted items flagged stale	Must	Stale changes shown with a warning and excluded
FLT-SIM-06	Quick "Can I afford…?" from home (amount and date only)	Should	Returns the verdict in one step
FLT-SIM-07	Describe a scenario in words ("client pays two weeks late"); AI drafts the changes for confirmation	Should	Defined in section 7; nothing runs until the user confirms the drafted changes

2.7 Insights (INS)

ID	Requirement	Priority	Acceptance criteria
FLT-INS-01	Rule-based insights, each with its numbers: negative before next income; large expenses clustered in one week; available below total because of reservations (breakdown); next income uncertain or overdue; recurring expense increased	Must	Each rule has fixtures that trigger it and fixtures that don't
FLT-INS-02	AI plain-language weekly summary of computed insights	Should	Section 7 rules apply

Cross-cutting edge cases

Edge case	Behaviour
Irregular income	Custom dates or one-time entries
Delayed income	Marked late with new date; forecast moves; insight names the effect
Income that never arrives	Marked missed; removed; history kept
Multiple currencies	Each account in its own currency; totals at stored rates; rates older than 7 days flagged
Recurring expenses	Month-end handling; pause and end dates
Missed payments	Overdue items stay on today until paid or skipped
Duplicate transactions	Import fingerprint flags likely duplicates
Transfers between accounts	Linked pair; net zero
Negative projected balance	Highlighted; Safe-to-Spend ₦0 with shortfall named
Changes to future commitments	Applied from the chosen date forward
Manual balance corrections	Adjustment transaction, never rewriting history
Timezone and travel	Dates anchored to the user's chosen timezone

3. User experience and interface

Float is designed for a 10-second daily check on a phone: open, see Safe-to-Spend and the next few items, confirm what happened, close. Desktop adds width for the timeline, chart and imports.

Information architecture

Home · Timeline (list and chart) · Plan (income, expenses, commitments) · What-if (scenarios) · Accounts (balances, transactions, imports) · Settings (currency, rates, buffer, notifications, data export)

Navigation

Viewport	Pattern
Phone	Fixed bottom navigation: Home, Timeline, Add (central quick entry for income, expense, transfer, commitment), Accounts, More (Forecast, What-if simulator, Categories, Settings)
Tablet and desktop	224 px left sidebar (Home; Money: Accounts, Transactions, Income, Expenses, Commitments; Planning: Cash-flow timeline, Forecast, What-if simulator; Personal: Categories, Settings) plus a top bar with quick add; timeline and chart side by side; current vs simulated plan in two columns

Onboarding

1. Sign in (magic link or Google); choose base currency and timezone.
2. Add accounts and balances, or import a CSV.
3. "When do you next expect money?" → first income source with certainty.
4. Pick recurring bills from a checklist (rent, electricity, internet, data, subscriptions, loan).
5. Reserve anything already set aside.
6. Set a buffer → first Safe-to-Spend with its breakdown highlighted. Progress dots show 6 steps; every step is skippable and resumable.

Primary journeys

J1 – Daily check: Home → Safe-to-Spend and per-day → "Today" card asks about due items ("Did your ₦8,000 subscription go out?") → Yes / Not yet / Skip → numbers update.

J2 – Can I afford it?: What-if → New → "One-off expense" → amount and date → comparison and verdict → try another date → discard, save or apply.

J3 – Late client: Timeline → tap expected income → Mark late → new date → Safe-to-Spend and warnings update; insight explains the effect.

J4 – Reconcile with bank: Accounts → account → "Update balance" → enter bank figure → difference shown with likely causes (unmarked items, duplicates) → confirm items or record adjustment.

J5 – Reserve for rent: Plan → Commitments → New → name, amount, accounts, link to rent expense → available drops; rent payment later consumes it.

Key screens

Screen	Purpose	Key behaviour
Home	Today's position	Safe-to-Spend, total · reserved · available, next 5 items, top insight, buffer warning
Safe-to-Spend breakdown	Trust	Step-by-step table with the lowest day highlighted and assumptions listed
Timeline	Money over time	Grouped by day or week; running balance; mode toggle; chart tab
What-if	Decisions	Scenario list; builder; comparison chart; verdict
Plan	Rules	Income, expenses and commitments with next occurrence and certainty
Account detail	Accuracy	Transactions, import history, balance update
Rates	Multi-currency	Each rate with date and source note

Mobile interaction patterns

- Timeline rows are cards with amount right-aligned and certainty chips; swipe to mark paid or received, with button equivalents.
- Chart switches to a simplified line with large tap targets for days; values read out in a bottom sheet.

- Quick add opens a 3-field bottom sheet (amount, what, when) with "More options".
- Number inputs use the numeric keypad and format naira as you type.
- Scenario builder is a step flow on phones; comparison uses stacked cards rather than side-by-side columns.

States

State	Treatment
Empty	Home before setup shows the onboarding checklist, not zeros
Loading	Skeleton numbers; the engine runs locally so recalculation never shows a spinner
Error	Inline validation; sync errors shown with retry
Offline	Read-only banner with last sync time; quick-add is queued
Warning	Buffer breach: amber card with date and amount; negative: red card with shortfall
Success	Brief confirmation and the number animating to its new value

Visual identity

- **Personality:** calm, optimistic, personal and grounded; spacious where Ledgerly is dense. Full specification: Float design system (DESIGN.md).
- **Colour:** deep teal primary (#235E62), sage (#75A99A) and soft sage (#DCEBE5) on warm ivory (#F8F7F1), charcoal text, a restrained warm accent (#D6AA69); dark theme is deep teal-charcoal (#152321). Status colours inform, never judge: available, planned, caution, shortfall, information. Actual, confirmed, estimated and simulated amounts are always visually distinct (solid, patterned, dashed).
- **Type:** DM Sans for the interface and all money figures (tabular numerals); DM Mono for calculations, references and technical dates. Icons: Lucide.
- **Themes:** light and dark.
- **Micro-interactions:** 150–250 ms; quick-add sheets slide in; the chart updates smoothly when a scenario changes. No animated money counters or celebratory effects; reduced motion respected.

4. Technical architecture

Float is a single Next.js application with a pure TypeScript forecast engine that runs in the browser for instant simulation and on the server for alerts; there is no separate API service and no queue.

Decisions

Area	Choice	Why
App	Next.js (App Router), React, TypeScript, Tailwind CSS, shadcn/ui	Shared stack
Forms and validation	React Hook Form, Zod (shared with API)	One schema per input
Data fetching and state	TanStack Query for server data; Zustand for scenario state	Scenarios live client-side until saved
Backend	Next.js route handlers	Light API; no separate service needed
Forecast engine	Pure TypeScript package: recurrence expansion, conversion, accumulation, Safe-to-Spend, scenarios	Same results in browser and server; property-testable
Recurrence	rrule.js (RFC 5545)	Proven rules including last working day
Dates	date-fns with date-fns-tz	Timezone-safe date arithmetic
Money	Integer minor units; decimal.js for rate multiplication	No floating-point errors
Database	PostgreSQL (Neon) with Drizzle ORM	Relational history; row-level security per user
Authentication	Auth.js (magic link, Google), optional TOTP two-factor	Shared stack; no passwords stored
Background jobs	Inngest: daily forecast run per user, buffer alerts, due-date reminders	Scheduled work without operating Redis; no heavy batch processing
Notifications	Web Push (VAPID) and email (Resend)	Alerts with an email fallback

Area	Choice	Why
Charts	visx	Custom projection, buffer band and scenario overlay that stay readable on phones
CSV parsing	Papa Parse in the browser	Raw files never uploaded; only parsed rows are sent
File storage	Not needed in the MVP	No uploads stored (CSV parsed client-side)
Realtime	Not needed	Single-user data; refetch on focus
AI (Should)	Anthropic Claude API with structured outputs	Explanations and scenario drafting only (section 7)
Caching	TanStack Query cache and a persisted snapshot in IndexedDB for offline reading	Instant home screen
Search	Postgres trigram on transaction descriptions	Find a transaction quickly
Logging and monitoring	pino, Sentry (amounts and descriptions scrubbed), web-vitals	Debuggable without leaking financial data
Analytics	PostHog with IP anonymisation; no amounts or descriptions captured	Privacy-conscious
Hosting	Vercel + Neon	Shared deployment pattern

API design

- REST via route handlers; Zod-validated; error shape { code, message, fields?, requestId }; cursor pagination on transactions (default 50).
- One snapshot endpoint gives the engine everything it needs; writes return the updated snapshot slice so cache and forecast refresh in one round trip.

Method	Path	Purpose
GET	/api/snapshot	Accounts, balances, rules, occurrences, commitments, rates, settings
GET, POST, PATCH, DELETE	/api/accounts	Accounts
POST	/api/accounts/:id/adjust	Balance correction
GET, POST, PATCH, DELETE	/api/transactions?account=&q=&cursor=	Transactions
POST	/api/transfers	Both sides of a transfer atomically
POST	/api/imports	Commit parsed and mapped rows with a file hash
GET, POST, PATCH, DELETE	/api/income, /api/expenses	Rules
PATCH	/api/occurrences/:ruleType/:ruleId/:date	Received, paid, late, skipped, missed
GET, POST, PATCH, DELETE	/api/commitments, /api/commitments/:id/reservations	Reservations
GET, POST	/api/fx-rates	Rates with date and source note
GET, POST, PATCH, DELETE	/api/scenarios; POST /api/scenarios/:id/apply	Scenarios
POST	/api/ai/explain, /api/ai/draft-scenario	AI (Should)
GET	/api/forecast?mode=&days=	Server-computed forecast (alerts, tests)
GET	/api/export; DELETE /api/account	Export and deletion

5. Database and domain model

Rules describe what's expected, occurrences record what happened to each expected instance, and transactions record real money; forecasts are always computed, never stored as truth.

```
erDiagram
  USER ||--o{ ACCOUNT : owns
  ACCOUNT ||--o{ TRANSACTION : records
  ACCOUNT ||--o{ IMPORT : receives
  IMPORT ||--o{ TRANSACTION : creates
  USER ||--o{ INCOME_SOURCE : expects
  USER ||--o{ EXPENSE_RULE : schedules
  INCOME_SOURCE ||--o{ OCCURRENCE : produces
  EXPENSE_RULE ||--o{ OCCURRENCE : produces
  OCCURRENCE ||--o| TRANSACTION : settled_by
  USER ||--o{ COMMITMENT : sets
  COMMITMENT ||--o{ RESERVATION : holds
  ACCOUNT ||--o{ RESERVATION : funds
  COMMITMENT }o--o| EXPENSE_RULE : pays
  USER ||--o{ FX_RATE : maintains
  USER ||--o{ SCENARIO : saves
  USER ||--o{ ALERT : receives
```

Entities

Entity	Key attributes	Constraints and notes
users	id, email, base_currency, timezone, buffer_minor, default_mode	
accounts	id, user_id, name, type, currency, include_in_forecast, archived_at	Balance derived from transactions
transactions	id, account_id, date, amount_minor, kind, category, description, occurrence_id, transfer_group_id, import_id, fingerprint	Index (account_id, date); trigram on description
imports	id, account_id, file_hash, row_count, mapping (JSONB)	Unique (account_id, file_hash)
fx_rates	id, user_id, from_currency, to_currency, rate (numeric 18,8), effective_date, source_note	

Entity	Key attributes	Constraints and notes
income_sources	id, user_id, name, account_id, amount_minor, amount_max_minor, rrule, start_date, end_date, certainty	
expense_rules	id, user_id, name, account_id, amount_minor, kind, category, rrule, start_date, end_date, paused	
occurrences	id, rule_type, rule_id, scheduled_date, status, actual_date, actual_amount_minor, override_amount_minor, override_certainty	Created lazily when the user acts; unique (rule_type, rule_id, scheduled_date)
commitments	id, user_id, name, target_minor, target_date, linked_expense_rule_id, released_at	
reservations	id, commitment_id, account_id, amount_minor, created_at	
scenarios	id, user_id, name, changes (JSONB), created_at	Changes validated by Zod
alerts	id, user_id, kind, data (JSONB), sent_at, dismissed_at	
ai_outputs	id, user_id, kind, input_hash, output, model, prompt_version, created_at	Cached explanations, labelled AI

Status fields

Field	Values
transactions.kind	income, expense, transfer_in, transfer_out, adjustment
occurrences.status	expected, received, paid, late, skipped, missed, overdue
income_sources.certainty	confirmed, estimated, uncertain
expense_rules.kind	bill, subscription, debt, variable, planned, one_time

Ownership, audit and retention

- **Ownership:** each user owns all their data; nothing is shared.
- **Audit:** balance corrections, applied scenarios and occurrence status changes are recorded with timestamps; transactions are never edited after reconciliation, only voided and replaced.
- **Retention:** kept until the user deletes it; raw CSV files are never stored; AI outputs cached 30 days; deleting the account removes all data within 24 hours.

6. PWA and mobile-first requirements

Float runs from its URL on any phone and installs to the home screen as a standalone app; the daily check works offline from the last synced snapshot.

ID	Requirement	Acceptance criteria
FLT-PWA-01	Manifest: name "Float", start_url /home, standalone, theme colour, maskable icons 192 and 512 px, shortcuts ("Add expense", "Can I afford…?")	Lighthouse installability passes; shortcuts appear on Android long-press
FLT-PWA-02	Service worker (Serwist) precaching the shell	App opens offline after first visit
FLT-PWA-03	Offline home, timeline and breakdown from a snapshot in IndexedDB, with "as of" time	Airplane-mode test shows the last Safe-to-Spend and its timestamp
FLT-PWA-04	Offline quick-add queued and synced	A queued expense appears after reconnect without duplicates
FLT-PWA-05	Offline fallback page for uncached routes	No browser error pages offline
FLT-PWA-06	Install prompt after onboarding completes; iOS guide	Shown once; available later from Settings
FLT-PWA-07	Safe-area handling for the bottom navigation and the central Add button	No overlap with home indicator
FLT-PWA-08	Responsive at 375, 768, 1024, 1440 px	No horizontal scroll at 375 px

Caching strategy

Resource	Strategy
Shell, JS, CSS, fonts, icons	Precache
Snapshot (/api/snapshot)	Network-first; copy persisted in IndexedDB for offline reading
Other GETs	Stale-while-revalidate, 24 h
Writes	Network, with an IndexedDB queue for quick-add and occurrence status changes
AI endpoints	Network-only

Sync: queued writes replay in order with idempotency keys; if a rule changed on another device meanwhile, the server rejects the write and the user sees both versions to choose.

Notifications: buffer alerts, due-date reminders and "did this happen?" prompts by Web Push, with email fallback.

Platform limitations and fallbacks

Capability	Android	iOS and iPadOS	Fallback
Install	Native prompt; app shortcuts supported	Add to Home Screen only; no shortcuts	Guide; quick actions inside the app
Push	Supported	Installed apps on iOS 16.4+ only	Email
Background Sync	Supported	Not supported	Replay on open, on online event, and every 60 s while open
Storage	Persistent storage can be requested	Possible eviction after weeks unused	Server is the source of truth; snapshot refreshed on open
Badging	Supported in installed apps	Supported for installed apps on recent iOS	In-app counts

7. AI and automation

AI helps people understand the forecast and explore scenarios in plain words; every number still comes from the deterministic engine, and AI never changes data without confirmation.

Deterministic vs AI

Operation	Owner	Why
Recurrence, conversion, projection, Safe-to-Spend, scenario results, verdicts	Engine (code)	Exact, testable, reproducible
Insight detection	Rules (code)	Numbers must be exact
Explaining why Safe-to-Spend is what it is	AI (Should)	Plain-language narrative of the computed breakdown
Turning a typed question into scenario changes	AI (Should)	Natural input ("what if Acme pays two weeks late?")
Weekly summary of insights	AI (Should)	Readable digest of computed facts

Use cases

ID	Use case	Input	Output
FLT-AI-01	Explain Safe-to-Spend or the forecast	The computed breakdown (items, lowest day, buffer, excluded income)	3–5 sentences, e.g. "Your lowest point is Nov 22 after rent and your loan. Your Nov 10 client payment isn't counted because it's marked estimated."
FLT-AI-02	Draft a scenario from text	User's sentence plus a list of the user's income and expense items (names, IDs, dates)	A structured list of scenario changes (Zod-validated) shown for confirmation
FLT-AI-03	Weekly summary	Computed insights for the week	Short paragraph with links to the items

Orchestration and safeguards

- Structured outputs with a strict schema; FLT-AI-02 can only reference item IDs that exist in the payload.
- **Number check in code:** every amount and date in AI text must appear in the input payload, or the text is discarded and the breakdown table is shown instead.
- **Human approval:** drafted scenario changes are listed with toggles; nothing runs until the user taps "Run scenario", and nothing is saved to real data without the existing apply confirmation.
- AI text is labelled "AI explanation" and sits beneath the computed breakdown.
- **Failure handling:** 10 s timeout, one retry, then the deterministic breakdown alone; unparseable drafts show "I couldn't turn that into changes, try the builder."
- **Limits and cost:** 30 AI requests per user per day; explanations cached by input hash for 30 days; a small model handles drafting.
- **Privacy:** only amounts, dates and item names are sent (no account numbers); users can turn AI off; provider terms exclude training on inputs.

8. Security, privacy and reliability

Float holds a person's complete financial picture, so it collects the minimum, keeps it private to one user, and makes leaving easy.

Area	Requirement
Authentication	Auth.js magic link and Google; optional TOTP two-factor; HTTP-only, Secure, SameSite=Lax cookies; 30-day idle expiry; sign out everywhere
Access control	Single-user ownership checked on every query; row-level security on user_id
Data isolation	Other users' IDs return 404; covered by the permission suite
Input validation	Zod on every endpoint; CSV parsed client-side and re-validated server-side; amounts bounded to ± ₦100 billion
Web vulnerabilities	CSRF protection, strict CSP, parameterised queries, dependency scanning
File handling	No files stored; CSV parsed in the browser

Area	Requirement
Rate limiting	Sign-in 5 per 15 min per email; API 120 requests per minute per user; AI per section 7
Encryption	TLS; encryption at rest by Neon; IndexedDB snapshot cleared on sign-out
Audit logging	Security events (sign-in, 2FA changes, export, deletion) logged per user
Backup and recovery	Neon point-in-time recovery 7 days; daily logical backup 30 days; RPO 24 h, RTO 4 h
Privacy controls	Full export (CSV and JSON); AI on/off; no third-party trackers; analytics never include amounts or names
Data deletion	Delete account removes all data within 24 hours, including backups after their retention window
Error monitoring	Sentry with amounts, descriptions and names scrubbed

9. Non-functional requirements

Category	Requirement	Target
Accuracy	Forecast totals vs timeline	Exact to the minor unit
Accuracy	Rounding	Banker's rounding once per conversion, documented
Performance	90-day forecast, 20 accounts, 200 rules, in the browser	< 50 ms
Performance	Home interactive on a mid-range Android over 4G	< 2 s
Dates	Timezone correctness	No item shifts day around midnight in the test matrix
Accessibility	WCAG 2.1 AA; charts have text equivalents; sign and label distinguish positive and negative	0 critical axe violations
Reliability	Availability	99.5% monthly

Category	Requirement	Target
Scalability	Per user	50 accounts, 500 rules, 50,000 transactions without degrading targets
Security	Cross-user access	0 in the permission suite
Maintainability	Engine coverage; strict typing	⩾ 95% engine line coverage; TypeScript strict
Browser support	Chrome, Edge, Firefox, Safari (last 2), Safari iOS 16.4+, Samsung Internet	E2E passes on Chromium, WebKit, Firefox
Mobile usability	Lighthouse mobile	Performance, accessibility, PWA ⩾ 90; touch targets ⩾ 44 px
Offline resilience	Home, timeline and breakdown	Open offline from the last snapshot

10. Testing strategy

The forecast engine is the product, so it gets property-based tests and a library of named real-life scenarios, starting with the worked example.

Type	What is tested	Tools
Unit	Recurrence for every schedule type, month ends, leap years, pauses; money and FX rounding	Vitest
Financial accuracy	Named scenarios with expected Safe-to-Spend (worked example, late income, missed income, negative dip, multi-currency, reserved rent)	Vitest snapshots
Property	Spending X today lowers every P(d) by exactly X; transfers never change totals; reservations never change total; an empty scenario equals the real forecast; adding confirmed income never lowers Safe-to-Spend	fast-check

Type	What is tested	Tools
Dates	Midnight and timezone edges with a fixed clock	Vitest fake timers
Integration	Occurrence status changes, transfers, imports, apply-scenario against Postgres	Vitest + Testcontainers
API	Schemas, pagination, error shapes, snapshot slices	Vitest
Permission	Every endpoint called as another user returns 404	Generated matrix
Data validation	Malformed CSVs, bad rates, extreme amounts	Vitest
AI	Draft-scenario outputs validated against schema; number check rejects invented figures (recorded responses)	Vitest
End-to-end	Onboarding to first Safe-to-Spend; simulate and apply; reconcile a balance	Playwright, phone and desktop
PWA and offline	Install checks; offline home from snapshot; queued quick-add syncs once	Lighthouse CI, Playwright network emulation
Responsive	Key screens at 375, 768, 1024, 1440 px	Playwright screenshots
Performance	Engine benchmark	Vitest bench (< 50 ms)

11. MVP and phased roadmap

The MVP delivers a correct, inspectable Safe-to-Spend Number and safe what-if testing on a phone; everything else is secondary.

Priority	Features
Must (MVP)	ACC-01–06; INC-01–04, 06; EXP-01–03; COM-01–04; FCT-01–10; SIM-01–05; INS-01; PWA-01–08; data export and deletion; seeded demo profile

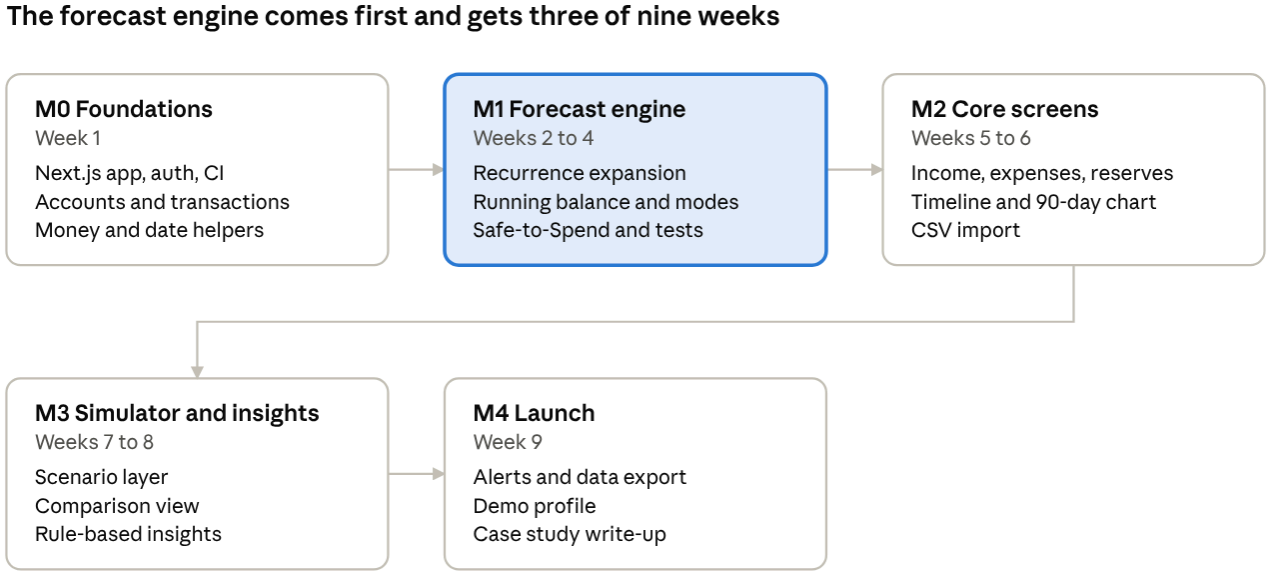
Priority	Features
Should (MVP if time)	Import templates (ACC-07), income ranges (INC-05), price changes (EXP-04), quick "Can I afford…?" (SIM-06), AI explanations, scenario drafting and weekly summary (SIM-07, INS-02, AI-01–03)
Could (Phase 2)	Goal-style reservations (COM-05), debt tracker, savings goals
Excluded from MVP	Bank connections, live FX feed, shared household plans, investment or tax advice, native apps

Phase 2: goal reservations, saved bank import templates for the major Nigerian banks, probability-weighted forecasts based on each client's history of late payments, integration with Debtfree so repayments flow into the forecast.

Phase 3: bank connections through Mono or Okra, live FX rates with source attribution, shared household plans.

12. Implementation planning

The engine is built and tested before the screens, so every screen is a view over numbers already known to be right.



MVP milestones · 5 phases, about 9 weeks

Week counts assume part-time solo work; the PWA shell and design tokens are set up in M0.

Sequence and dependencies

Milestone	Delivers	Depends on
M0 Foundations	Next.js app, Auth.js, CI, accounts and transactions, money and date helpers, PWA shell	—
M1 Forecast engine	Recurrence, conversion, accumulation, modes, Safe-to-Spend, scenario layer, full test suite	M0 helpers
M2 Core screens	Income, expenses, commitments, timeline, chart, breakdown, CSV import	M1
M3 Simulator and insights	Scenario builder, comparison, verdicts, insights; AI Should-haves if time	M2
M4 Launch	Alerts, export and deletion, offline snapshot, demo profile, case study	M0–M3

Technical risks and early validation

Risk	Early validation
Recurrence and timezone bugs	Spike rrule.js with Africa/Lagos and month-end rules in week 1
Engine too slow for live simulation	Benchmark 200 rules × 90 days in the browser in M1
Users find setup too long	Paper-test the 6-step onboarding with 3 people before building it
iOS offline and push limits	Test install, offline snapshot and push on a real iPhone in M0

Definition of Done

- Acceptance criteria for the requirement IDs pass in automated tests; engine changes add a named scenario.
- Phone and desktop layouts checked; axe clean; empty, loading, error, offline and success states present.
- Every displayed figure traceable to the engine; no amounts in logs or analytics.

Demo data

A seeded freelancer, "Tolu": naira current account, savings and a USD domiciliary account; four income sources of mixed certainty (two clients, a retainer, a one-off); rent reserved for the year; a realistic month of bills, subscriptions and a loan; one late payment and two saved scenarios ("New laptop", "Client pays late"). A guest login opens it.

Deployment

- **Environments:** local, preview per pull request (Vercel + Neon branch), production.
- **Configuration:** Zod-validated environment variables; Inngest, VAPID and AI keys as secrets.
- **CI:** lint, type-check, engine and property tests, integration, permission and E2E suites, Lighthouse budgets.
- **Migrations:** Drizzle migrations applied to the preview branch in CI, then on production deploy.